

Geometric concepts in CG

Below is a **clean, well-structured, detailed, and properly formatted explanation** of all the concepts you provided. Nothing is omitted; everything is reorganized into a coherent flow suitable for notes or exam preparation.

Basic Elements of Computer Graphics

Computer Graphics (CG) deals with representing and manipulating geometric objects such as **lines**, **polygons**, and **polyhedra**. In a three-dimensional world, these objects possess certain measurable properties like **length**, **angles**, and **orientation**.

A **polyhedron** is a 3D geometric solid with:

- Flat faces
- Straight edges
- Sharp vertices

Examples: cube, tetrahedron, prism.

To mathematically define and operate on geometric objects, CG uses three fundamental types:

1. **Points**
 2. **Scalars**
 3. **Vectors**
-

Geometric View: Points, Scalars, and Vectors

1. Points

- A **point** is the most fundamental geometric entity.
 - In 3D space, a point represents a **location**.
 - It has **no size, no width, no shape, and no dimension**.
 - Points alone are insufficient to describe geometry; we need numbers to define relations such as distances.
-

2. Scalars

- A scalar is a **quantity that has magnitude only** (size), with no direction.
- Scalars obey all operations of regular arithmetic (addition, subtraction, multiplication, division).
- They follow associative and commutative rules.

- Common examples:
 - Real numbers and complex numbers
 - Temperature

Scalars are essential for measuring lengths, scaling vectors, and defining parameters in equations.

3. Vectors

- A vector has **two attributes**:
 1. **Magnitude** (length)
 2. **Direction** (orientation)
 - Examples:
 - Force
 - Velocity
 - A vector **does not have a fixed position**; it represents displacement.
 - In computer graphics, a **directed line segment** with length and direction represents a vector.
-

Geometric Lines

Definition

A line in an affine space is formed using points and vectors.

There are two equivalent ways to express a line:

1. **Point + Vector**
2. **Point - Point** (difference of coordinates gives a direction vector)

Parametric Form of a Line

A line can be represented as:

$$P(\alpha) = P_0 + \alpha d$$

where:

- P_0 = a fixed point on the line
- d = direction vector
- α = scalar parameter

Interpretation

- When $\alpha = 0$, the point on the line is P_0 .
- As α increases or decreases, $P(\alpha)$ moves along the direction of vector d .

- If $\alpha \geq 0$, the set of points forms a **ray** starting from P_0 .
- For all real values of α , the set of points forms an **infinite line**.

Types of Geometric Line Structures

1. Line

- Extends infinitely in both directions.

2. Line Segment

- A finite portion of a line between two endpoints.

3. Ray

- Starts at one point and extends infinitely in one direction.

Line-Drawing in Raster Graphics

To draw a line on a raster display (pixel grid), the system performs these steps:

1. Convert the line's endpoints into integer screen coordinates.
2. Determine which pixel positions are closest to the mathematical line path between the endpoints.
3. Store the line's color in the frame buffer at those pixel coordinates.
4. The video controller reads the frame buffer and lights up the appropriate pixels.

Because the display is discrete, a rasterized line is only an **approximation** of the actual continuous line.

Sampling in Line Drawing

- Raster lines are drawn using **sampling**.
- The line is evaluated at unit intervals along one axis, and the nearest integer coordinate on the other axis is selected.
- Since pixels are discrete, perfect continuity cannot be achieved.

What Is an Ideal Line?

An ideal line drawn on a raster display should satisfy:

1. Straightness and visual continuity

- Perfect straightness is achievable only for horizontal, vertical, and 45° lines.

2. Endpoint interpolation

- The drawn line must pass through the exact endpoints.

3. Uniform density and intensity

- Pixel distribution should appear even.
- Variations cause visible irregularities.

4. Consideration of aliasing and anti-aliasing

- **Aliasing**: jagged edges on diagonal or curved lines caused by pixel discretization.
- Enlarging a bitmap makes aliasing more visible.
- **Anti-aliasing** smooths these jagged edges by adjusting pixel intensities.

5. Efficiency

- Line drawing must be fast because many lines may need to be drawn per frame.

Simple Line Equation

The Cartesian slope-intercept form of a line is:

$$y = mx + b$$

where:

- m = slope
- b = y-intercept

Simple Digital Line-Drawing Approach

- Increment x pixel-by-pixel.
- Compute corresponding y using the formula $y = mx + b$.

This simple method is conceptually easy but not efficient or accurate for raster systems, which led to more advanced algorithms like **DDA** and **Bresenham's algorithm**.

If you want, I can rewrite this in **textbook style**, **exam-ready points**, or **short notes format**.

Below is a **complete, clean, properly formatted, deeply explained** version of everything you provided about **DDA** and **Bresenham's Line Drawing Algorithm**.

All points, formulas, steps, examples, advantages, and code are included. Nothing is omitted or altered.

Line-Drawing Algorithms

Computer graphics systems must convert mathematically defined lines into a set of **discrete pixels** on a raster display. Two widely used algorithms for this are:

1. **Digital Differential Analyzer (DDA) Algorithm**
2. **Bresenham's Line Drawing Algorithm**

1. Digital Differential Analyzer (DDA)

Concept

DDA is a **scan-conversion algorithm** that uses the line's slope-intercept equation:

$$y = mx + b$$

For endpoints (x_0, y_0) and (x_1, y_1) :

$$m = \frac{y_1 - y_0}{x_1 - x_0}$$

$$b = y_0 - mx_0$$

For any increment in x by δx , the corresponding increment in y is:

$$\delta y = m \cdot \delta x$$

DDA Algorithm Principle

- The line is **sampled at unit intervals**.
- If the slope $m \leq 1$, x increases by 1 each step and y is computed using:

$$y_{k+1} = y_k + m$$

- If the slope $m > 1$, y increases by 1 and x is computed using:

$$x_{k+1} = x_k + \frac{1}{m}$$

- At each step, x or y is **rounded to the nearest integer pixel**.

Case Handling in DDA

For slope $m > 1$:

- Sample at unit steps in y ($dy = 1$).
- Compute:

$$x_{k+1} = x_k + \frac{1}{m}$$

For slope $m \leq 1$:

- Sample at unit steps in x ($dx = 1$).
- Compute:

$$y_{k+1} = y_k + m$$

In each step:

- Increment k by 1
- Round computed x or y to nearest integer.

DDA Program (As Given)

```
cpp

inline int round (const float a) {
    return int (a + 0.5);
}

void lineDDA (int x0, int y0, int xEnd, int yEnd)
{
    int dx = xEnd - x0, dy = yEnd - y0, steps, k;
    float xIncrement, yIncrement, x = x0, y = y0;

    if (fabs(dx) > fabs(dy))
        steps = fabs(dx);
    else
        steps = fabs(dy);

    xIncrement = float(dx) / float(steps);
    yIncrement = float(dy) / float(steps);

    setPixel(round(x), round(y));

    for (k = 0; k < steps; k++) {
        x += xIncrement;
        y += yIncrement;
        setPixel(round(x), round(y));
    }
}
```

Disadvantages of DDA

1. Uses **floating point arithmetic**, which is expensive.
2. Requires **rounding** at every step.
3. More computation per pixel than desired for hardware implementation.

This motivates the need for Bresenham's algorithm.

2. Bresenham's Line Drawing Algorithm

Key Characteristics

- Extremely efficient and accurate.
- Uses **ONLY integer arithmetic** (addition, subtraction, comparison).
- Faster than DDA because it avoids multiplication and division.
- Based on a **decision parameter** determining the best pixel at each step.

The algorithm is explained for lines where:

$$0 < m < 1$$

But it generalizes to all 8 octants using symmetry.

Core Idea of Bresenham

At each step in x (x increases by 1), the algorithm must choose between two possible pixels:

- **Pixel A:** (x_{k+1}, y_k)
- **Pixel B:** $(x_{k+1}, y_k + 1)$

To decide which pixel is closer to the true line, a **decision parameter** p_k is used.

Mathematical Derivation (Simplified but Complete)

True line equation:

$$y = mx + b$$

At step k , the algorithm evaluates distances:

- **Lower pixel deviation**

$$d_{\text{lower}} = y_{\text{line}} - y_k$$

- **Upper pixel deviation**

$$d_{\text{upper}} = (y_k + 1) - y_{\text{line}}$$

The algorithm checks the sign of:

$$d_{\text{lower}} - d_{\text{upper}}$$

If negative → choose the lower pixel

If positive → choose the upper pixel

Decision Parameter

After simplification and eliminating floating-point calculations, the decision parameter is:

$$p_0 = 2\Delta y - \Delta x$$

Update rules:

- If $p_k < 0$: choose lower pixel

$$p_{k+1} = p_k + 2\Delta y$$

- If $p_k \geq 0$: choose upper pixel

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

Bresenham Algorithm for $|m| < 1$

Steps

1. Input endpoints and place left endpoint into (x_0, y_0) .
2. Plot the first point.
3. Compute:
 - $\Delta x = x_1 - x_0$
 - $\Delta y = y_1 - y_0$
 - Decision parameter:

$$p_0 = 2\Delta y - \Delta x$$

4. For each x from x_0 to x_1 :
 - If $p_k < 0$:
 - Next point is (x_{k+1}, y_k)
 - Update: $p_{k+1} = p_k + 2\Delta y$
 - Else:
 - Next point is $(x_{k+1}, y_k + 1)$
 - Update: $p_{k+1} = p_k + 2\Delta y - 2\Delta x$
5. Repeat until $x = x_1$.

Example

Draw line from (20,10) to (30,18):

$$\Delta x = 10, \quad \Delta y = 8$$

$$p_0 = 2\Delta y - \Delta x = 16 - 10 = 6$$

$$2\Delta y = 16, \quad 2\Delta y - 2\Delta x = -4$$

At each step, check p_k and decide whether y increases.

This example demonstrates how the plotted points follow the line with minimal error.

Code for Bresenham's Algorithm (Given)

cpp

```
void lineBres (int x0, int y0, int xEnd, int yEnd)
{
    int dx = fabs(xEnd - x0), dy = fabs(yEnd - y0);
    int x, y, p = 2*dy - dx;
    int twoDy = 2*dy, twoDyMinusDx = 2*(dy - dx);

    if (x0 > xEnd) {
        x = xEnd; y = yEnd; xEnd = x0;
    } else {
        x = x0; y = y0;
    }

    setPixel(x, y);

    while (x < xEnd) {
        x++;
        if (p < 0)
            p += twoDy;
        else {
            y++;
            p += twoDyMinusDx;
        }
        setPixel(x, y);
    }
}
```

Advantages of Bresenham's Algorithm

1. Only **integer arithmetic** is used.
2. No floating-point rounding.
3. Very fast; implementable in hardware.
4. Avoids duplicate points.
5. More efficient than DDA for raster displays.

Disadvantages

1. Designed for **basic line drawing**; does not directly support anti-aliasing.
2. Initialization step must be handled externally.
3. Extensions are required for thick or smooth lines.

Handling All Octants

The algorithm can be adapted based on the slope:

- $m > 1 \Rightarrow$ swap x and y
- $m < 0 \Rightarrow$ decrement y instead of increment
- $m < -1 \Rightarrow$ both adjustments above

This ensures the algorithm draws lines correctly in **all 8 octants**.

Use in Circle Drawing

Bresenham's incremental logic is also used in:

- Midpoint Circle Algorithm
- Drawing 8 symmetric radii at angles 0°, 45°, 90°, 135°, etc.

Horizontal, vertical, and $\pm 45^\circ$ lines are treated as special, trivial cases.

If you want, I can produce:

- ✓ A neatly summarized exam-ready short notes version
- ✓ A side-by-side comparison table of DDA vs Bresenham
- ✓ A visual diagram showing the decision choices in each step

Just tell me what format you prefer.

Scan Converting Circles, Midpoint Algorithms, and Area Filling — complete, exam-ready explanation

This note brings together the slide content and the images you provided into a single, complete, correctly-formatted explanation. Every key formula, decision rule, algorithm step and advantage/disadvantage is included.

1. Basic approaches to rasterizing a circle

A circle with center (x_c, y_c) and radius R satisfies the implicit equation

$$(x - x_c)^2 + (y - y_c)^2 = R^2.$$

Three common ways to generate pixels for a circular arc / full circle:

1. Explicit (solve $y = f(x)$)

Solve for y as a function of x :

$$y = y_c \pm \sqrt{R^2 - (x - x_c)^2}.$$

- To draw a quarter circle you can iterate x from x_c to $x_c + R$, compute y and round to the nearest pixel.
- **Drawbacks:** expensive square-root computations each step, non-uniform pixel spacing (gaps near shallow/steep parts); poor visual uniformity.

2. Parametric (step angle)

Use parametric form:

$$x = x_c + R \cos \theta, \quad y = y_c + R \sin \theta,$$

and step θ from 0 to $\pi/2$ for a quarter circle.

- **Advantages:** avoids solving square roots per x-step; sampling by angle gives better geometric spacing than explicit $y = f(x)$ in some regions.
- **Drawbacks:** requires trigonometric functions (sin, cos) or costly incremental trig approximations; choosing the angular increment is nontrivial to get uniform pixel spacing.

3. Implicit / midpoint style

Work with the implicit circle function

$$F(x, y) = (x - x_c)^2 + (y - y_c)^2 - R^2.$$

- If $F = 0$: point on the circle; $F < 0$: inside; $F > 0$: outside.
- Use a midpoint test between candidate pixels to decide which pixel is closer to the true curve. This leads to the **Midpoint Circle Algorithm** (an integer-only incremental method), which is the circle analogue of Bresenham's line algorithm. It is efficient and gives uniform-looking pixels.

2. Exploiting symmetry: 8-way symmetry of a circle

A circle is symmetric across the axes and diagonals. If you calculate a pixel (x, y) in the first octant (where $0 \leq x \leq y$ for the upper-right octant), you can reflect it to produce up to 8 pixels:

$$(x, y), (y, x), (-x, y), (-y, x), (-x, -y), (-y, -x), (x, -y), (y, -x)$$

This reduces computation: compute points only for the first octant then write the eight symmetric pixels for the full circle.

Typical helper function (pseudocode):

```
c
void CirclePoints(int x, int y, int value) {
    WritePixel( x,  y, value);
    WritePixel( y,  x, value);
    WritePixel( y, -x, value);
    WritePixel( x, -y, value);
    WritePixel(-x, -y, value);
    WritePixel(-y, -x, value);
    WritePixel(-y,  x, value);
    WritePixel(-x,  y, value);
}
```

3. Midpoint Circle Algorithm — derivation and integer implementation

We generate points in the first octant from $(0, R)$ down to where $x \geq y$. At each step we increment x by 1 and must choose whether to keep the same y (move **E**) or decrement y (move **SE**). We test the midpoint between those two candidate pixels.

Let the implicit function (centered at origin for simplicity) be

$$F(x, y) = x^2 + y^2 - R^2.$$

At step k the midpoint to test (when choosing between E and SE) is

$$M = (x_k + 1, y_k - \frac{1}{2}).$$

Compute

$$p_k = F(M) = (x_k + 1)^2 + (y_k - \frac{1}{2})^2 - R^2.$$

Decision:

- If $p_k < 0 \rightarrow$ midpoint is inside the circle $\rightarrow$ choose **E** (next pixel $(x + 1, y)$).
- If $p_k \geq 0 \rightarrow$ midpoint is outside or on circle $\rightarrow$ choose **SE** (next pixel $(x + 1, y - 1)$).

To avoid real arithmetic, derive integer update increments. Standard incremental updates yield:

• **Initialization:**

$$x = 0, \quad y = R, \quad p_0 = 1 - R$$

(this integer form is obtained by evaluating and rounding; some floating derivations show $p_0 = \frac{5}{4} - R$ but the integer algorithm uses $1 - R$.)

• **Loop (while $x < y$):**

- If $p < 0$: choose E
 - $x \leftarrow x + 1$
 - $p \leftarrow p + 2x + 1$ (common form: $p+ = 2x + 3$ if x is previous value; both are equivalent depending on when x incremented)
- Else: choose SE
 - $x \leftarrow x + 1$
 - $y \leftarrow y - 1$
 - $p \leftarrow p + 2x - 2y + 1$ (or $p+ = 2(x - y) + 5$ depending on update ordering)

Using the widely-used canonical update (increment x first, with initial $p = 1 - R$):

```
c
x = 0;
y = R;
p = 1 - R;
CirclePoints(x, y, value);

while (x < y) {
    x = x + 1;
    if (p < 0) {
```

```

    // choose E
    p = p + 2*x + 1;
} else {
    // choose SE
    y = y - 1;
    p = p + 2*(x - y) + 1;
}
CirclePoints(x, y, value);
}

```

(Implementation variants differ slightly in constants depending on whether the update uses previous x or incremented x ; these are algebraically equivalent.)

Properties and advantages

- Only integer arithmetic (adds, subtracts, comparisons). Good for hardware.
- Uses 8-way symmetry — compute 1/8th of the circle and reflect.
- Fast, visually uniform pixel distribution.

Note about the initial p_0 : in floating derivations $p_0 = \frac{5}{4} - R$; integer implementations commonly use $p_0 = 1 - R$ (equivalent after rounding).

4. Midpoint updates: reasoning summary (concise)

- Evaluate F at midpoint between candidate pixels — sign tells inside/outside.
- Precompute incremental differences so each iteration only performs integer additions.
- Two update cases: E or SE; each has a fixed integer increment expression for p .
- Stop when $x \geq y$ (first octant done).

5. Midpoint Ellipse Algorithm (outline)

Ellipse implicit:

$$F_{\text{ellipse}}(x, y) = \frac{x^2}{a^2} + \frac{y^2}{b^2} - 1$$

(or written in integer scaled form depending on r_x, r_y). The midpoint ellipse algorithm divides the first quadrant into **two regions**:

- **Region 1:** slope magnitude ≤ 1 (x-step major): iterate x and decide whether to decrement y .
- **Region 2:** slope magnitude > 1 (y-step major): iterate y and decide whether to increment x .

Procedure (summary):

1. Start at $(0, r_y)$.
2. Compute the initial decision parameter for region 1.
3. While in region 1 (stop condition depends on comparing $2b^2x$ and $2a^2y$):
 - If $p_1 < 0$: next point $(x + 1, y)$, update p_1 by adding incremental term.

- Else: next point $(x + 1, y - 1)$, update p_1 accordingly.
4. Compute initial parameter for region 2 from last point from region 1.
 5. While in region 2:
 - If $p_2 < 0$: next point $(x + 1, y - 1)$ or similar depending on sign, update p_2 .
 - Else: next point $(x, y - 1)$, update p_2 .
 6. Use 4-way symmetry (or 8-way if reflecting across both axes as appropriate) to plot other quadrants.

Full equations are longer (see slide formulas). The key idea is the same as circle: test a midpoint and update using integer increments. The ellipse algorithm must handle two regions because curvature changes.

6. Comparison: explicit / parametric / midpoint

- **Explicit ($y=f(x)$):** simple but non-uniform and expensive (sqrt).
- **Parametric:** uniform in angle, expensive trig per point or expensive incremental trig approximations.
- **Midpoint (Bresenham-style):** integer-only, fast, uniform-looking results, exploits symmetry and a midpoint decision test.

7. Area fill (filled primitives) — general ideas and algorithms

Filling a closed polygon (or any closed shape) means identifying and coloring all interior pixels.

Requirements for fill:

- The boundary must be closed and well-defined.
- A test must exist to determine whether a pixel is inside or outside.
- A method to visit and color all interior pixels.

Two major groups of algorithms:

A. Seed/Seed-based fills (recursive or iterative)

Assume a single known interior pixel (seed).

1. Flood-fill (also called seed fill)

- Replace all connected pixels of a target color (e.g., background or “unfilled”) with the fill color.
- Variants: 4-connected and 8-connected.

Pseudocode (recursive 4-connected):

```
c
void flood_fill_4(int x, int y, Color target, Color fill) {
    if (read_pixel(x,y) != target) return;
    write_pixel(x,y, fill);
```

```
flood_fill_4(x+1, y, target, fill);
flood_fill_4(x-1, y, target, fill);
flood_fill_4(x, y+1, target, fill);
flood_fill_4(x, y-1, target, fill);
}
```

8-connected adds diagonals ($x \pm 1, y \pm 1$) as additional recursive calls.

Pros

- Simple to implement.
- Fills complex shapes regardless of boundary color (fills any connected region of target color).

Cons

- Recursive form may cause stack overflow for large regions.
- Inefficient for big areas (many repeated checks) and can be slow.
- Memory/time cost can be high; iterative stack/queue versions are preferred in practice.
- If using color replacement, must ensure the fill target is correct (avoid replacing boundary).

2. Boundary-fill

- Starts at a seed and replaces target pixels until a boundary color is encountered.
- Two variants: 4-connect and 8-connect.
- Pseudocode similar to flood_fill but test is `if pixel != boundary_color && pixel != fill_color`.

Example (4-connected boundary fill):

```
c

void boundary_fill4(int x, int y, Color fill, Color boundary) {
    Color c = read_pixel(x,y);
    if (c != boundary && c != fill) {
        write_pixel(x,y,fill);
        boundary_fill4(x+1,y,fill,boundary);
        boundary_fill4(x-1,y,fill,boundary);
        boundary_fill4(x,y+1,fill,boundary);
        boundary_fill4(x,y-1,fill,boundary);
    }
}
```

Pros

- Works when boundary color is known and distinct.
- Fills interiors by searching outward from seed until boundary detected.

Cons

- Same recursion/stack and efficiency issues as flood-fill.
- Requires boundary to be a single consistent color (problematic for anti-aliased edges or multi-color boundaries).

B. Scan-line Fill (recommended for polygons)

- More efficient and commonly used for polygon filling.
- For each scan line (row of pixels), compute intersections of that scanline with polygon edges; sort intersection x-coordinates; fill spans between pairs of intersections.
- Works well for simple and complex polygons, more memory-efficient and avoids recursion.
- Also supports color interpolation across spans (see below).

8. Fill connectivity: 4-connect vs 8-connect

- **4-connected** neighbors: up, down, left, right. Avoids diagonal connectivity.
- **8-connected** neighbors: includes diagonals — up-left, up-right, down-left, down-right.
- Choice affects which pixels are considered connected and thus which interior pixels get filled. Use 8-connected to fill diagonal adjacency.

9. Practical considerations & optimizations for fill

- Use **iterative** stack/queue implementations instead of recursion to avoid stack overflow.
- Use scan-line seed filling (span filling) to reduce repeated work: find x-extents of contiguous fill runs on a scanline, push neighbor spans to stack/queue.
- For polygons, **edge table / active edge table** method (scan-line fill) is asymptotically efficient: for each scanline maintain active edges, compute intersection x, fill between pairs.
- For anti-aliased shapes, boundary colors are not solid; use scanline algorithms or stencil buffers in hardware.

10. Color interpolation across spans (scanline shading)

When polygons have vertex colors (Gouraud shading) or a gradient, you interpolate color across a span:

- For a span between X_{left} and X_{right} on a scanline, compute color at endpoints by interpolating along edges, then linearly interpolate colors for pixels across the span.
- Notation: vertices C_1, C_2, C_3, C_4 — interpolate along edges to get colors at scanline endpoints then interpolate across span.
- Used in software rasterizers and implemented by fixed-point incremental arithmetic for performance.

11. Recap of algorithms and when to use them

- **Midpoint circle:** use for drawing circles/ellipses efficiently on integer grids; hardware-friendly and visually uniform.
- **Parametric / explicit methods:** fine for small or approximate plotting or when trig hardware is cheap; not recommended for integer rasterization.

- **Ellipse midpoint:** analogous to circle but with two regions; use for integer raster ellipses.
- **Flood-fill / boundary-fill:** simple for small regions and teaching; use iterative or span-based implementations in production.
- **Scan-line polygon fill:** recommended for polygon interior filling (efficient, deterministic, works with interpolation).

12. Example: midpoint circle numerical example (quick)

Circle radius $R = 10$. Initialize:

- $x = 0, y = 10, p = 1 - R = -9$.
- Plot symmetric points for $(0, 10)$.
- Iterate x from 0 to y (while $x < y$), update p and y according to decision test; continue plotting symmetric points.

(If you want, I can produce the full list of plotted coordinates for $R = 10$.)

13. Code snippets (compact, ready to copy)

Midpoint circle (canonical integer variant):

```
c
void MidpointCircle(int xc, int yc, int R, int color) {
    int x = 0;
    int y = R;
    int p = 1 - R;
    CirclePoints(xc + x, yc + y, color);

    while (x < y) {
        x++;
        if (p < 0) {
            // choose E
            p += 2*x + 1;
        } else {
            // choose SE
            y--;
            p += 2*(x - y) + 1;
        }
        CirclePoints(xc + x, yc + y, color);
    }
}
```

Boundary-fill (4-connect, recursive):

```
c
void BoundaryFill4(int x, int y, Color fill, Color boundary) {
    Color c = read_pixel(x, y);
    if (c != boundary && c != fill) {
        write_pixel(x, y, fill);
        BoundaryFill4(x+1, y, fill, boundary);
        BoundaryFill4(x-1, y, fill, boundary);
        BoundaryFill4(x, y+1, fill, boundary);
        BoundaryFill4(x, y-1, fill, boundary);
    }
}
```

```
}
}
```

Flood-fill (8-connect, iterative using stack):

```
c

void FloodFill8(int sx, int sy, Color target, Color fill) {
    if (read_pixel(sx, sy) != target) return;
    stack.push((sx,sy));
    while (!stack.empty()) {
        (x,y) = stack.pop();
        if (read_pixel(x,y) != target) continue;
        write_pixel(x,y,fill);
        for each (dx,dy) in [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(-1,1),(1,-1),(1,1)]
            stack.push((x+dx,y+dy));
    }
}
```

If you want, I can now:

- produce a **one-page cheat sheet** that you can print for exam revision, or
- **generate a step-by-step trace** (list of plotted pixels) for a specific radius like R=10 to see how the midpoint algorithm works numerically, or
- create **visual diagrams** (images) that show the midpoint test and the 8-way reflections.

Tell me which of those you'd like and I'll produce it right away.

Below is a single, complete, carefully organized, exam-ready explanation that collects **every important concept, formula, rule, algorithm, special case and code fragment** shown in your slides and images. I do not omit any of the key points — I explain the window→viewport pipeline, normalization and coordinate transforms, scan-line polygon fill (including special vertex cases and an implementation), line clipping (Cohen–Sutherland and Liang–Barsky with intersection math and a worked example), polygon clipping and concavity tests, and related practical notes (zoom/pan/aspect ratio, normals, polygon tables, inside/outside tests). Code excerpts from the slides are included and annotated.

1 — 2D Viewing pipeline (window → viewport → device)

Purpose. Convert world coordinates of scene objects into coordinates used by the display device. Typical stages:

1. **Model coordinates → World coordinates**
(modeling transforms build the scene)
2. **World coordinates → Viewing coordinates**
(positioning camera / view transform)
3. **Viewing coordinates → Normalized coordinates**
(map clipping window to canonical normalized rectangle, usually $[0, 1] \times [0, 1]$)

4. Normalized coordinates → Device coordinates (viewport)

(scale and translate to actual pixel rectangle on screen)

Window vs Viewport

- **Window (clipping window):** region in the world/viewing coordinate space that is selected for display (also called clipping window).
- **Viewport:** rectangular region on the display (device coordinates) where the window contents are mapped.
- Graphics systems commonly use rectangular clipping windows aligned with axes. Non-rectangular clipping needs extra code.

Zooming and Panning

- **Zoom:** map a smaller part of the world window onto the viewport → zoom in. Map a larger window → zoom out.
- **Panning:** move the fixed-size clipping window across the world to see different parts of the scene.

Aspect ratio

- Maintain object proportions only if viewport aspect ratio equals window aspect ratio (i.e. scale factors s_x and s_y are equal). If not equal, objects appear stretched.

2 — Normalization and window→viewport coordinate transform

Given a clipping window defined by $[x_{wmin}, x_{wmax}] \times [y_{wmin}, y_{wmax}]$ and a viewport $[x_{vmin}, x_{vmax}] \times [y_{vmin}, y_{vmax}]$:

Normalized coordinates (unit square):

$$x_n = \frac{x_w - x_{wmin}}{x_{wmax} - x_{wmin}}, \quad y_n = \frac{y_w - y_{wmin}}{y_{wmax} - y_{wmin}}$$

so points in the window map to $[0, 1] \times [0, 1]$.

Map normalized to viewport:

$$x_v = x_{vmin} + x_n (x_{vmax} - x_{vmin}) = x_{vmin} + \frac{(x_w - x_{wmin})(x_{vmax} - x_{vmin})}{x_{wmax} - x_{wmin}}$$

$$y_v = y_{vmin} + \frac{(y_w - y_{wmin})(y_{vmax} - y_{vmin})}{y_{wmax} - y_{wmin}}$$

Two-step view transform (equivalent):

1. Scale about fixed point (often lower-left) by $S(s_x, s_y)$.
2. Translate lower-left corner of window to lower-left corner of viewport with translation T .

3 — Scan-Line Polygon Fill (concept and algorithm)

Goal. Fill the interior of (possibly concave) polygon(s) by drawing horizontal spans for each scan line.

Basic idea.

- For each scan line (integer y), compute intersections of the scan line with polygon edges.
- Sort intersection x-values left-to-right.
- Pair successive intersections into spans $[x_i, x_{i+1}]$, and fill inside spans using the odd-even rule (fill every other segment).

Odd-Even (Parity) Rule.

- Draw a ray from point to infinity and count how many edges it crosses. If number of crossings is odd $\Rightarrow$ point is inside; even $\Rightarrow$ outside.
- For scan fill, intersections appear in even count (for non-degenerate cases); pairs (1st-2nd, 3rd-4th) are interior spans.

Algorithm (high level)

1. Intersect each scanline with all polygon edges.
2. Sort intersections by x.
3. Compute parity/in-out and fill "in" pixels between intersection pairs.
4. Special handling:
 - Exclude horizontal edges (they do not contribute to crossing count).
 - Handle scanlines passing through vertices correctly (see vertex rules below).
5. Use coherence: intersections for scanline y are similar to $y + 1$ (incrementally update x intersections using inverse slope).

Important numeric/implementation detail

Edge slope is constant between scanlines; you can incrementally update x-intersection by adding $1/m$ (the inverse slope) between successive scanlines. Use integer arithmetic or fixed-point for efficiency.

4 — Vertex / scanline intersection special cases

Consider a scanline passing exactly through a polygon vertex where two edges meet. Two cases:

1. **Vertex is local extremum** (both adjacent edges lie to same side of scanline; both either above or below). Then the scanline intersects polygon at a single point but the two edges are both either entering or leaving; treat the vertex intersection as a single intersection (count once).
2. **Vertex is not an extremum** (one adjacent edge above, other below). Then scanline crosses both edges at the vertex; count as two intersections (i.e., treat normally).

Rule to decide programmatically

Trace polygon vertices in consistent order (clockwise or counterclockwise). For consecutive vertex y-values y_{i-1}, y_i, y_{i+1} :

- If y_{i-1}, y_i, y_{i+1} are monotonic (all increasing or all decreasing), count vertex once.

- Otherwise count vertex twice (as two intersections).

Common implementation trick

Adjust the top y of each edge (subtract 1) so that shared vertices are not double-counted. Typical step: when processing an edge, decrement the y coordinate of the edge's upper endpoint by 1 (so that scanline equality ties are resolved consistently).

5 — Practical integer scan-fill implementation (slide code explained)

The slide shows a classic approach with left edge (`le[]`) and right edge (`re[]`) arrays:

`scanfill(...)` core

- Prepare arrays `le[y] = +infty` and `re[y] = -infty` for each y (scanline).
- For each polygon edge call `edgedetect` to fill `le` and `re` with min and max x for each scanline intersected by that edge.
- After processing edges, for each y: if `le[y] <= re[y]` then fill pixels `i = le[y]` to `re[y]-1`.

`edgedetect(x1,y1,x2,y2, le, re)` (edge rasterization into scanline min/max)

1. If $y_2 < y_1$ swap endpoints so loop runs upward in y.
2. Compute slope inverse $mx = (x_2 - x_1) / (y_2 - y_1)$ (if $y_2 - y_1 == 0$ set $mx = x_2 - x_1$ to avoid divide by zero).
3. Initialize $x = x_1$.
4. For $i = y_1$ to y_2 inclusive:
 - if $x < le[i]$ then $le[i] = \text{floor}(x)$
 - if $x > re[i]$ then $re[i] = \text{floor}(x)$ (or cast to int)
 - $x += mx$

`draw_pixel(x,y)` draws point, using `glBegin(GL_POINTS)` etc. (slide code included).

This method is simple and efficient for convex or concave polygons if vertex cases and horizontal edges are handled properly.

6 — Scanline algorithm summary and optimizations

- Complexity: For polygon with E edges and H scanlines, naive approach is $O(E \cdot H)$. More efficient is **Edge Table / Active Edge Table** which processes edges once and removes/activates as y changes (typical polygon rasterizer).
- Use incremental arithmetic for x-intersections to avoid per-scanline division.
- Exclude horizontal edges from edge table or handle them specially.
- Use span filling and stack-based span flood to avoid recursion for area fills.

7 — Polygon connectivity, concavity, and splitting

Concave polygon identification

- A polygon is concave if it has at least one interior angle $> 180^\circ$ or if extension of some edges intersects other edges.
- One test: compute cross product (z component) of adjacent edge vectors for each vertex when vertices are ordered consistently (counter-clockwise).
 - For counter-clockwise ordering:
 - Convex vertex $\rightarrow$ cross product (z) > 0
 - Concave vertex $\rightarrow$ cross product (z) < 0
- If the sign differs at any vertex the polygon is concave; you can split concave polygon into convex pieces along an appropriate diagonal.

Rotational method

- Rotate polygon so each vertex sits at origin in turn. If the following vertex lies below x-axis, the polygon is concave at that vertex — split along x-axis from that vertex.

Splitting strategy

- When a concavity detected (negative cross product), split polygon along a diagonal that resolves self-intersection or concavity. Many libraries triangulate polygons to manage concavity and rendering.

8 — Polygon data structures (polygon tables)

Store polygon geometry and attributes:

- **Geometric data:** vertex list, edge lists, slopes, bounding boxes.
- **Attribute data:** color, transparency, material properties.
- For rendering meshes, storing shared edges or edge lists avoids drawing shared edges twice and supports adjacency queries.

Front/Back face orientation

- Use right-hand rule (counter-clockwise winding) to define outward-pointing normal.
- OpenGL uses face orientation for backface culling — outward-facing if vertices listed counter-clockwise in screen space.

Normal vector

- For triangle with vertices V_1, V_2, V_3 :

$$\mathbf{N} = (V_2 - V_1) \times (V_3 - V_1)$$

- Direction indicates front face orientation.

9 — Inside-outside tests (point in polygon)

Odd-Even rule: Draw a ray from point P to infinity and count crossings of polygon edges. Odd $\Rightarrow$ inside, even $\Rightarrow$ outside.

Winding number rule: Compute winding number by summing oriented edge contributions; non-zero $\Rightarrow$ inside. Useful for complex polygons and possible holes.

10 — Line clipping: Cohen-Sutherland algorithm

Purpose. Efficiently clip a line segment to an axis-aligned rectangular window.

Outcodes. Assign a 4-bit code (TOP, BOTTOM, RIGHT, LEFT) to each endpoint indicating which half-space(s) the point is outside of.

Bit assignment (example: MSB to LSB):

- bit 3: TOP ($y > y_{\max}$)
- bit 2: BOTTOM ($y < y_{\min}$)
- bit 1: RIGHT ($x > x_{\max}$)
- bit 0: LEFT ($x < x_{\min}$)

Algorithm

1. Compute outcodes `code1` and `code2`.
2. If $(\text{code1} \mid \text{code2}) == 0 \rightarrow$ both inside $\Rightarrow$ trivially accept.
3. Else if $(\text{code1} \& \text{code2}) \neq 0 \rightarrow$ share an outside region $\Rightarrow$ trivially reject (both endpoints outside on same side).
4. Else: choose an endpoint outside (non-zero code), find intersection with corresponding window boundary, update endpoint and outcode, repeat until accept or reject.

Intersection computations

- Parametric line: $y = y_0 + m(x - x_0)$, slope $m = (y_1 - y_0)/(x_1 - x_0)$.
- With vertical boundaries:
 - left: $x = x_{\min} \rightarrow y = y_0 + m(x_{\min} - x_0)$
 - right: $x = x_{\max}$
- With horizontal boundaries:
 - bottom: $y = y_{\min} \rightarrow x = x_0 + (1/m)(y_{\min} - y_0)$
 - top: $y = y_{\max}$

Slide code snippet (conceptual) shows drawing of original line and window and calling `CohenSutherlandLineClipAndDraw`.

11 — Liang-Barsky line clipping (parametric, more efficient)

Idea. Use parametric form of line $x = x_0 + t \cdot dx$, $y = y_0 + t \cdot dy$, $0 \leq t \leq 1$. For each clip edge produce an inequality of the form $pt \leq q$. Solve to get entry/leaving parameters t_{entry} , t_{leave} .

Define:

- $dx = x_1 - x_0$, $dy = y_1 - y_0$.
- For each edge compute (p, q) :
 - Left: $p = -dx$, $q = x_0 - x_{min}$
 - Right: $p = dx$, $q = x_{max} - x_0$
 - Bottom: $p = -dy$, $q = y_0 - y_{min}$
 - Top: $p = dy$, $q = y_{max} - y_0$

For each edge:

- If $p == 0$ and $q < 0 \rightarrow$ parallel and outside $\Rightarrow$ reject.
- If $p < 0$ (potential entry): $t = q/p$; if $t > t_{entry}$ set $t_{entry} = t$; if $t > t_{leave}$ reject early.
- If $p > 0$ (potential leaving): $t = q/p$; if $t < t_{leave}$ set $t_{leave} = t$; if $t < t_{entry}$ reject early.

If not rejected:

- If $t_{entry} > 0$: set $x'_0 = x_0 + t_{entry} \cdot dx$, $y'_0 = y_0 + t_{entry} \cdot dy$.
- If $t_{leave} < 1$: set $x'_1 = x_0 + t_{leave} \cdot dx$, $y'_1 = y_0 + t_{leave} \cdot dy$.
- Draw clipped segment between t_{entry} and t_{leave} .

Advantages: fewer intersection calculations than Cohen-Sutherland; operates with parametric values.

Slide example (numerical)

- Given $dx=250$, $dy=140$, window $[xmin=70, xmax=230, ymin=60, ymax=150]$, endpoints $P0=(30,20)$, $P1=(280,160)$:
 - Left edge: $p=-250$, $q=30-70=-40 \rightarrow t=0.16 \Rightarrow t_{entry} = 0.16$
 - Right edge: $p=250$, $q=230-30=200 \rightarrow t=0.8 \Rightarrow t_{leave} = 0.8$
 - Bottom: $p=-140$, $q=20-60=-40 \rightarrow t \sim 0.2857 \Rightarrow$ update $t_{entry} = 0.2857$
 - Top: $p=140$, $q=150-20=130 \rightarrow t \sim 0.928 \Rightarrow t_{leave}$ remains 0.8 ($0.928 > 0.8 \Rightarrow$ no change)
 - Final $t_{entry}=0.2857$, $t_{leave}=0.8$. New clipped endpoints computed using these t values produce final clipped segment (given numerically in slides).

Slide code provides `cliptest()` and `LiangBarskyLineClipAndDraw()` implementation and viewport mapping.

12 — Cohen-Sutherland vs Liang-Barsky — short comparison

- **Cohen-Sutherland:**
 - Simple bit-code tests for trivial accept/reject quickly.
 - Iterative clipping with intersection math when needed.
 - Good when many lines are trivially accepted/rejected.
- **Liang-Barsky:**
 - Parametric approach computing t_{entry} and t_{leave} .
 - Can require fewer intersection calculations than Cohen-Sutherland for non-trivial clips.
 - Slightly more math but efficient.

13 — Polygon clipping: Sutherland-Hodgman

Problem. Clipping a polygon requires producing clipped polygon(s) that remain closed polylines; clipping each edge separately (line clipping) would leave disconnected segments.

Sutherland-Hodgman algorithm (vertex-by-vertex)

- Clip polygon against each window edge in turn (left, right, bottom, top).
- For each edge:
 - Traverse polygon vertices in order; for each edge (S, P) :
 - If P is inside and S is inside $\rightarrow$ output P.
 - If P is inside and S is outside $\rightarrow$ output intersection I, then P.
 - If P is outside and S is inside $\rightarrow$ output intersection I.
 - If both outside $\rightarrow$ output nothing.
- After processing all edges, result is clipped polygon (may be multiple vertices). Works correctly for **convex clipping windows** (axis-aligned rectangle) and convex subject polygons; for concave subject polygons Sutherland-Hodgman still produces correct clipping but may generate extraneous edges, and with concave clipping windows additional care is required.

Note: Sutherland-Hodgman is commonly used because it preserves polygon order and produces closed polygons ready for filling.

14 — Filling algorithms summary (seed-fill vs scanline)

Seed-fill (Flood/Boundary fill):

- Start from a seed pixel inside region.
- **Flood-fill:** replace all connected pixels of target color with fill color (4- or 8-connected). Can be recursive or iterative using stack/queue.
- **Boundary-fill:** fill until a boundary color is reached (requires single uniform boundary color).
- Pros: conceptual simplicity.
- Cons: recursion depth, large memory/time cost for big regions, inefficient relative to scanline.

Scanline polygon fill:

- Build edge table, maintain active edge table, compute intersections per scanline, fill spans by pairs.
- Pros: efficient, stable, supports color interpolation across spans (Gouraud shading), widely used in rasterizers.
- Cons: slightly more complex to implement than naive flood fill.

15 — Inside/outside and face orientation (normals)

- **Inside test:** use parity (odd-even) or winding number.
- **Polygon facing:** compute cross product of two edges (winding order) to determine front/back face. OpenGL and other systems use this for backface culling and shading decisions.

16 — Practical OpenGL examples from the slides

- The slides include several program snippets showing integration of clipping/filling with OpenGL:
 - `display()` draws the polygon or line and calls `scanfill(...)` or clipping routine.
 - `scanfill(...)` prepares `le[]` and `re[]`, calls `edgedetect` for each polygon edge then draws pixels for each scanline from `le[y]` to `re[y]`.
 - `edgedetect(...)` computes the x intersection for each y and updates left/right arrays, using incremental $mx = (x2-x1)/(y2-y1)$ update.
 - `draw_pixel(...)` uses `glBegin(GL_POINTS)` and `glVertex2i(x,y)` to draw.
- Cohen–Sutherland demo: `ComputeOutCode()`, trivial accept/reject tests, then iterative clipping and then viewport mapping and drawing.
- Liang–Barsky demo: `cliptest(p,q,&te,&tl)` helper and `LiangBarskyLineClipAndDraw(...)` which computes `t_entry/ t_leave` and maps final clipped segment to viewport coordinates (scale and translate) before drawing viewport rectangle and clipped line.

17 — Tips and gotchas (summary for exam / implementation)

- Always handle horizontal and vertical edges specially in scanline algorithms.
- Use consistent vertex ordering (CW or CCW) for parity rules and face orientation.
- Decrement upper y endpoint by 1 (or otherwise adjust) to avoid double counting shared vertices.
- For numerical stability and speed, use integer or fixed-point arithmetic in raster loops where possible.
- When using viewport mapping, maintain aspect ratio by choosing viewport with same aspect ratio as window or apply uniform scaling.
- When clipping many lines, choose a method that favors your workload: Cohen–Sutherland for many trivial accept/rejects; Liang–Barsky when parametric efficiency helps.

- For polygon clipping, Sutherland–Hodgman is straightforward for convex clip windows; triangulation and robust libraries are recommended for complex cases.

18 — If you want worked traces / code variations

I can immediately produce any of the following (pick one and I will show it right away):

- A **step-by-step trace** of scanline intersection pairs and filled spans for the polygon in one of your slide figures (show exact intersections and filled pixels).
- A **worked trace** of the Liang–Barsky numeric example showing each p,q,t update (already summarized above, but I can expand with intermediate arithmetic).
- A **detailed step trace** of the midpoint-circle or ellipse algorithm for a particular radius showing every plotted pair and p updates.
- A **one-page printable cheat sheet** that compresses formulas, algorithms, and code snippets into a single A4 for exam revision.
- Clean, modern **C/C++ implementations** (edge table/active edge table polygon fill, Cohen–Sutherland, Liang–Barsky) with comments and no OpenGL dependencies.

Tell me which extra output you want and I'll produce it now (I won't ask clarifying questions — I'll assume you want one of those and produce it).